

1. Prerequisites to run this project:

- Java JDK 17;
- Maven 3.8.5 (or higher);

How to check if your system has Maven and Java installed: (source: <https://docs.spring.io/spring-boot/docs/current/reference/html/getting-started.html>)

- Open a terminal;
- Run the following command (java -version) to ensure that you have a valid version of Java installed:

```
java -version
openjdk version "17.0.4.1" 2022-08-12 LTS
OpenJDK Runtime Environment (build 17.0.4.1+1-LTS)
OpenJDK 64-Bit Server VM (build 17.0.4.1+1-LTS, mixed mode, sharing)
```

- Run the following command (mvn -v) to ensure that you have a valid version of Maven installed:

```
mvn -v
Apache Maven 3.8.5 (3599d3414f046de2324203b78ddcf9b5e4388aa0)
Maven home: usr/Users/developer/tools/maven/3.8.5
Java version: 17.0.4.1, vendor: BellSoft, runtime:
/Users/developer/sdkman/candidates/java/17.0.4.1-librca
```

(If you are missing any of the previous software, you can find a step-by-step on how to install Java and Maven in the last section of this document)

2. Running the project:

- After extracting the file ClassesBookings.zip, open a terminal on it's path location;

```
cd C:\PATH_TO_FOLDER\ClassesBookings
```

- Then run the following command: `mvn integration-test spring-boot:run`;

```
IdeaProjects\ClassesBookings> mvn integration-test spring-boot:run
```

The app should compile and start running as depicted below:

```
[INFO] Running com.example.ClassesBookings.ClassesBookingsApplicationTests
03:41:17.794 [main] INFO org.springframework.test.context.support.AnnotationConfigContextLoaderUtils -- Could not detect default configuration classes for test class [com.example.ClassesBookings.ClassesBookingsApplicationTests]: ClassesBookingsApplicationTests does not declare any static, non-private, non-final, nested classes annotated with @Configuration.
03:41:17.883 [main] INFO org.springframework.boot.test.context.SpringBootTestContextBootstrapper -- Found @SpringBootConfiguration com.example.ClassesBookings.ClassesBookingsApplication for test class com.example.ClassesBookings.ClassesBookingsApplicationTests

  ____  _
 / ___|| | | |
| |___| |_| | |
 \___ \|  _/ | | |
      | | | | | |
      |_| |_| |_|_|_|

:: Spring Boot ::
(v3.1.0)

2023-06-03T03:41:18.237+01:00 INFO 15808 --- [
  raula\IdeaProjects\ClassesBookings]
2023-06-03T03:41:18.238+01:00 INFO 15808 --- [
  main] c.e.C.ClassesBookingsApplicationTests : No active profile set, falling back to 1 default profile: "default"
2023-06-03T03:41:19.135+01:00 INFO 15808 --- [
  main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port(s): 0 (http)
2023-06-03T03:41:19.146+01:00 INFO 15808 --- [
  main] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2023-06-03T03:41:19.147+01:00 INFO 15808 --- [
  main] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/10.1.8]
2023-06-03T03:41:19.242+01:00 INFO 15808 --- [
  main] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring embedded WebApplicationContext
2023-06-03T03:41:19.244+01:00 INFO 15808 --- [
  main] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization completed in 979 ms
2023-06-03T03:41:20.133+01:00 INFO 15808 --- [
  main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port(s): 8080 (http) with context path ''
2023-06-03T03:41:20.141+01:00 INFO 15808 --- [
  main] c.e.C.ClassesBookingsApplicationTests : Started ClassesBookingsApplicationTests in 2.118 seconds (process running for 2.831)
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended
2023-06-03T03:41:20.867+01:00 INFO 15808 --- [o-auto-1-exec-2] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring DispatcherServlet 'dispatcherServlet'
2023-06-03T03:41:20.867+01:00 INFO 15808 --- [o-auto-1-exec-2] o.s.web.servlet.DispatcherServlet : Initializing Servlet 'dispatcherServlet'
2023-06-03T03:41:20.868+01:00 INFO 15808 --- [o-auto-1-exec-2] o.s.web.servlet.DispatcherServlet : Completed initialization in 1 ms
[INFO] Tests run: 19, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 3.435 s - in com.example.ClassesBookings.ClassesBookingsApplicationTests
[INFO] Results:
[INFO] Tests run: 19, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] --- jar:2.4:jar (default-jar) @ ClassesBookings ---
[INFO]
[INFO] --- spring-boot:3.1.0:repackage (repackage) @ ClassesBookings ---
[INFO] Replacing main artifact C:\Users\raul\IdeaProjects\ClassesBookings\target\ClassesBookings-0.0.1-SNAPSHOT.jar with repackaged archive, adding nested dependencies in BOOT-INF/.
```

The app is now live:

```
Tomcat started on port(s): 8080 (http) with context path ''
Started ClassesBookingsApplication in 1.446 seconds (process running for 1.665)
```

Now all that's left to do is navigate to <http://localhost:8080/swagger-ui/index.html>

localhost:8080/swagger-ui/index.html

Swagger Powered by SMARTBEAR

Glofox Tech Task 1.0 OAS3

/v3/api-docs

Created by Raul Bartolo

Servers

gym-class-controller

GET /classes Gets classes with optional search parameters

POST /classes Posts classes, use same start and end date to create a single class or a date interval to create multiple, subsequent classes. There can only be one class per day

DELETE /classes Deletes an existing class on a given date

booking-controller

POST /bookings Creates a booking in a given date for existing classes

DELETE /bookings Deletes a booking of a member in a given date for existing classes

Schemas

3. Comments:

I'd like to point out a few things about this development process that might help during testing:

Request Methods:

I've created a few extra methods, namely, for both `"/classes"` and `"/bookings"`:

`"/classes"`:

GET - With optional search parameters; Filters classes by name / date;

DELETE - Deletes classes by date;

`"/bookings"`:

DELETE – Deletes a booking by name (member's name) and a date;

I feel these methods provide a more robust solution (with more options and use-cases), with minimal added effort (both controllers also implement the expected, respective, POSTs requests).

Validations:

Validations were used to protect the REST API application from requests containing invalid data / containing data that was not compatible with implemented business logic. Some of the validations used:

- Verify that required parameters were filled;
- Verify that dates had a valid format and that the end date was not previous to the start date (I've used the format dd-MM-yyyy for this two stories) ;
- Verify that there's no existing class booked in for the date of the class we're booking; - Assumed there's only a class a day;
- Verify that the class/booking to delete exists;
- Verify that there's no existing booking, for the same member and same date of the booking we're attempting to create;

Unit Testing:

I've implemented unit testing to validate available requests creating assertions to evaluate both http status codes and expected response body.

Documentation:

It was also implemented a Swagger URL (<http://localhost:8080/swagger-ui/index.html>) for this API to help document, test and consume the RESTful web services.

4. Installing Java JDK and Maven:

(The following guide applies to systems running Windows. For other systems there's documentation available online on how to install Java JDK and Maven)

4.1 Java JDK:

Source for Java: <https://www.geeksforgeeks.org/download-and-install-java-development-kit-jdk-on-windows-mac-and-linux/>

Step 1: Download and Install Java Development Kit (JDK 17)

The very first step is to download the **Oracle Java Development Kit (JDK)** from the Official Oracle Website.

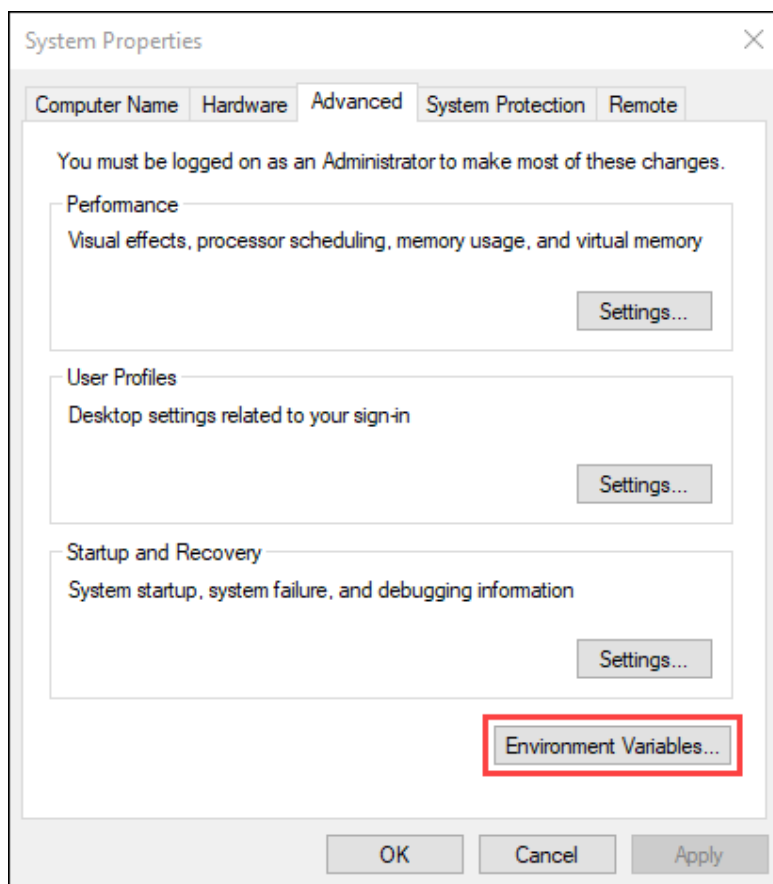
<https://www.oracle.com/java/technologies/downloads/#jdk17-windows>

Step 2: Configure Environment Variables

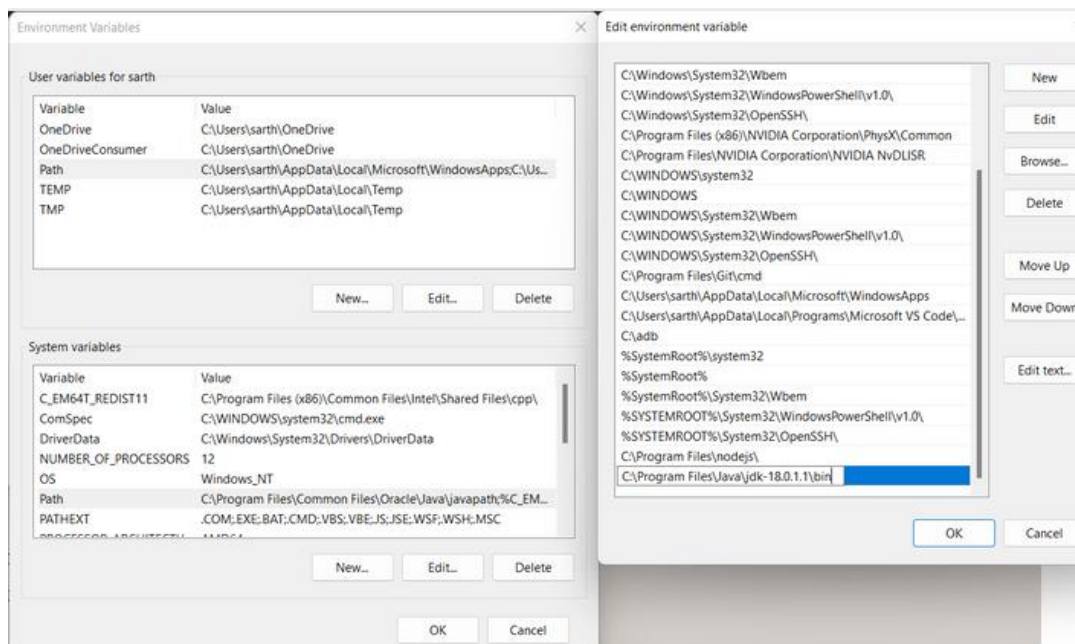
After the installation is complete, we have to configure environment variables to notify the system about the directory in which JDK files are located. Proceed to **C:\Program Files\Java\jdk-{YOUR_JDK_VERSION}\bin** (replace {-} with your JDK version)

To set the Environment Variables, you need to search Environment Variables in the Task Bar and click on **“Edit the system environment variables”**.

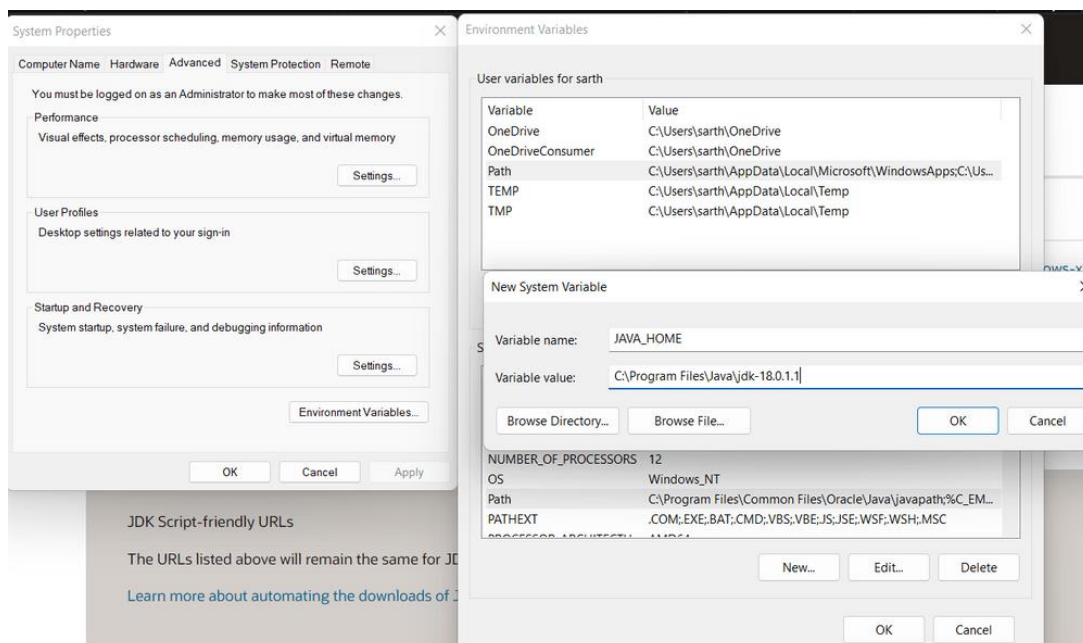
Under the **Advanced** section, Click on **“Environment Variables”**.



Under **System variables**, select the **“Path”** variable and click on **“Edit”**. Click on **“New”** then paste the Path Address i.e. **C:\Program Files\Java\jdk-{YOUR_JDK_VERSION}\bin**. Click on **“OK”**.



Now, in the **Environment Variables** dialogue, under **System variables**, click on “**New**” and then under **Variable name: JAVA_HOME** and **Variable value:** paste address i.e. **C:\Program Files\Java\jdk-{YOUR_JDK_VERSION}**. Click on **OK => OK => OK**.



Step 3: Restart your machine;

4.2 Maven:

Source for Maven: <https://phoenixnap.com/kb/install-maven-windows>

Step 1: Download Maven Zip File and Extract

Visit <https://maven.apache.org/download.cgi> and download the version of Maven you want to install. The *Files* section contains the archives of the latest version.

Centre

Maven >

Central Repository

Maven >

Developer Centre

Books and Resources

Security

COMMUNITY

Community Overview

Project Roles

How to Contribute

Getting Help

Files

Maven is distributed in several formats for your convenience. Simply pick a ready-made binary distribution archive and follow the [installation instructions](#). Use a source archive if you intend to build Maven yourself.

In order to guard against corrupted downloads/installations, it is highly recommended to [verify the signature](#) of the release bundles against the public [KEYS](#) used by the Apache Maven developers.

	Link	Checksums	Signature
Binary tar.gz archive	apache-maven-3.8.4-bin.tar.gz	apache-maven-3.8.4-bin.tar.gz.sha512	apache-maven-3.8.4-bin.tar.gz.asc
Binary zip archive	apache-maven-3.8.4-bin.zip	apache-maven-3.8.4-bin.zip.sha512	apache-maven-3.8.4-bin.zip.asc
Source tar.gz archive	apache-maven-3.8.4-src.tar.gz	apache-maven-3.8.4-src.tar.gz.sha512	apache-maven-3.8.4-src.tar.gz.asc
Source zip archive	apache-maven-3.8.4-src.zip	apache-maven-3.8.4-src.zip.sha512	apache-maven-3.8.4-src.zip.asc

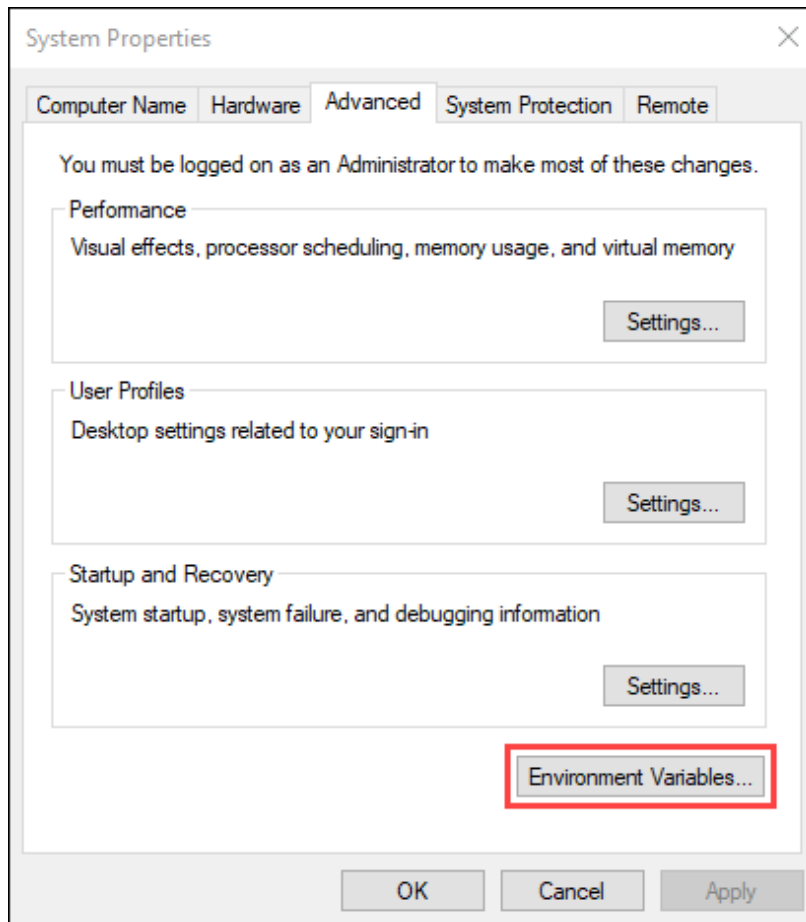
Click on the appropriate link to download the binary zip archive of the latest version of Maven. As of the time of writing, that is version 3.9.2.

Since there is no installation process, extract the Maven archive to a directory of your choice once the download is complete. Ex. *C:\Program Files\Maven\apache-maven-3.9.2*.

Step 2: Add MAVEN_HOME System Variable

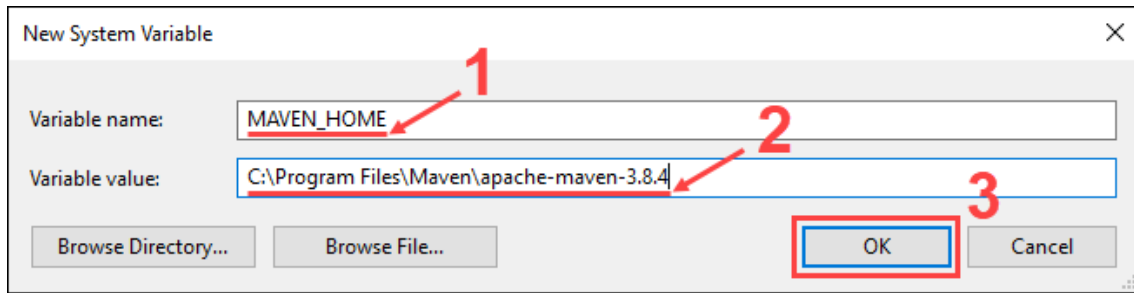
To set the Environment Variables, you need to search Environment Variables in the Task Bar and click on **“Edit the system environment variables”**.

Under the *Advanced tab* in the *System Properties* window, click **Environment Variables**.



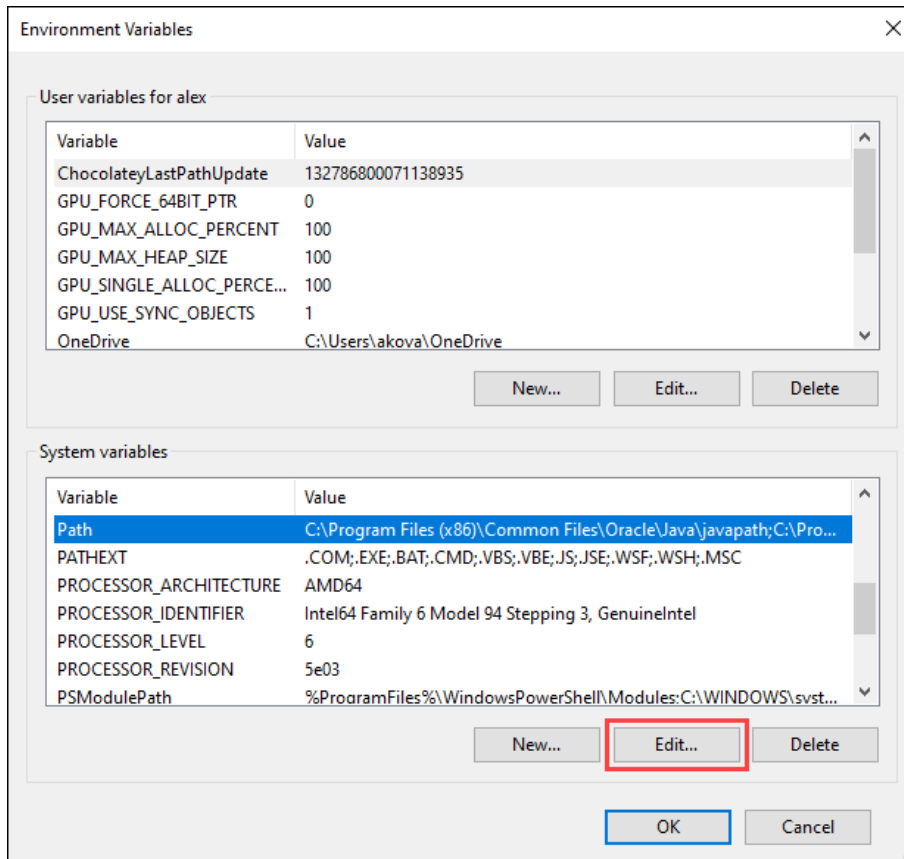
Click the **New** button under the *System variables* section to add a new system environment variable.

Enter **MAVEN_HOME** as the variable name and the path to the Maven directory as the variable value. Click **OK** to save the new system variable.



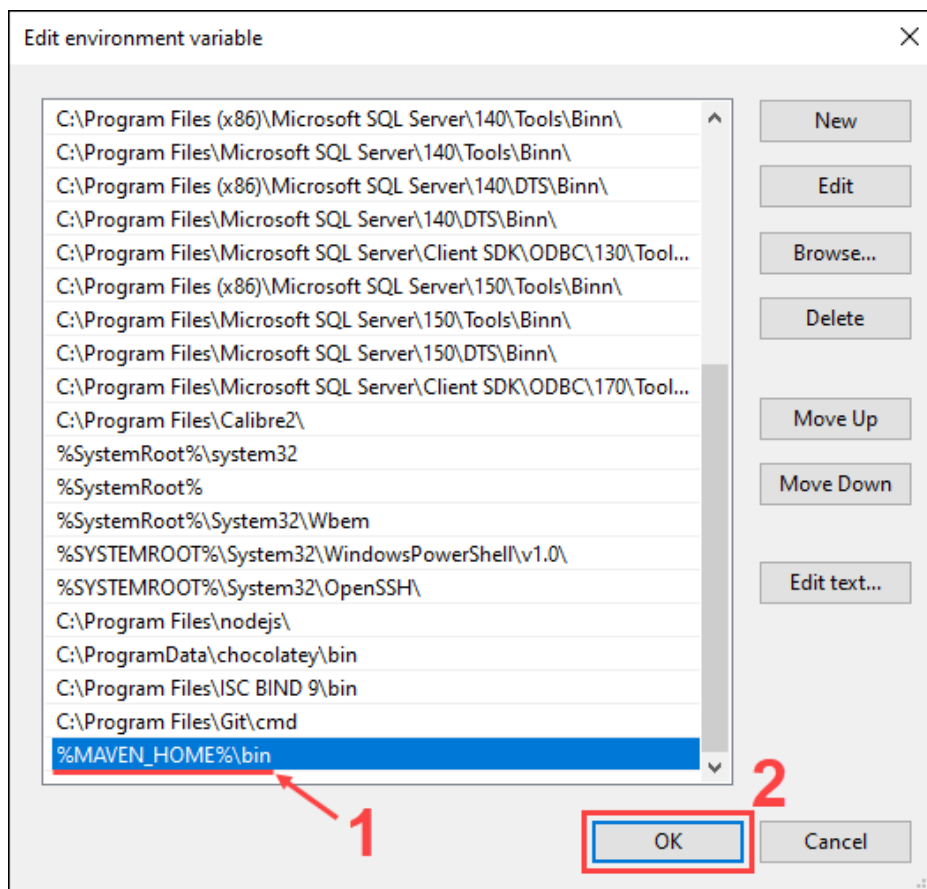
Step 3: Add MAVEN_HOME Directory in PATH Variable

Select the **Path** variable under the *System variables* section in the *Environment Variables* window. Click the **Edit** button to edit the variable.



Click the **New** button in the *Edit environment variable* window.

Enter **%MAVEN_HOME%\bin** in the new field. Click **OK** to save changes to the **Path** variable.



Click **OK** in the *Environment Variables* window to save the changes to the system variables.

Step 4: Restart your machine;